

Yocto Recipe Flow — A 10-Recipe Learning Path

A guided sequence through `poky/meta` recipes, ordered so each one reuses concepts from the previous step and adds one or two new ones. By recipe 10 (`linux-yocto`), every mechanic in the kernel recipe should be recognizable because you saw it first in a smaller, simpler recipe.

Repo used for all paths below: `/home/dipesh/projects/yocto/poky`

Study method for every step: 1. Read the `.bb / .inc` file top to bottom. 2. Open every `.bbclass` it `inherits` and find the task functions it defines. 3. Run `bitbake-getvar <recipe> -f do_<task>` to see the fully resolved shell function after all classes/appends are applied. 4. Run `bitbake -e <recipe> | less` and search for a variable you're unsure about — the output shows exactly which file set it last.

Recipe 1 — `makedevs` (absolute basics)

File: `meta/recipes-devtools/makedevs/makedevs_1.0.1.bb`

Why this one first: it has **zero** `inherit`, one local source file (no network fetcher noise), and a hand-written `do_compile / do_install`. This is the build system with nothing hidden — pure `base.bbclass` underneath.

What's in it: - `SRC_URI = "file://makedevs.c"` — a local file, no git/http fetcher yet. - `do_compile()` calls `${CC} ${CFLAGS} ${LDFLAGS}` directly. - `do_install()` — plain `install -d / install -m .` - `do_install:append:class-native_sdk()` — your first override syntax. - `BBCLASSEXTEND = "native native_sdk"`.

By the end of this step you should understand: - The five task names in a build: `fetch` → `unpack` → `patch` → `configure` → `compile` → `install` → `package` (which ones this recipe skips and why). - What `${S}`, `${D}`, `${WORKDIR}` mean and how they differ. - That `${CC}`, `${CFLAGS}`, `${LDFLAGS}` are supplied by the build system (cross-compiler variables), not hardcoded. - The `task:append:OVERRIDE()` syntax and what `BBCLASSEXTEND` does (build `native/native_sdk` variants of the same recipe). - That every recipe implicitly inherits `base.bbclass`, which is what provides the *default* `fetch/unpack/patch` tasks you don't see written anywhere.

Recipe 2 — `zlib` (a real build, still hand-written)

Files: `meta/recipes-core/zlib/zlib_1.3.1.bb`

Same shape as recipe 1 (hand-written `do_configure` / `do_compile` / `do_install`), but now the source is fetched from upstream and patched, and there's a real out-of-tree build directory.

What's new here: - `SRC_URI` pulls a tarball **and** applies a `.patch` file — see the patch step actually do something for the first time. - `B = "${WORKDIR}/build"` — build directory distinct from source dir `S`. - `inherit ptest + do_install_ptest()` — a runtime self-test package. - `RDEPENDS:${PN}-ptest` — a runtime dependency scoped to one sub-package.

By the end of this step you should understand: - How `do_patch` applies files listed in `SRC_URI` before `do_configure` runs, and where to look if a patch fails to apply. - Why `s` (source) and `B` (build) are sometimes different directories. - What `ptest.bbclass` adds: a `-ptest` package, `do_install_ptest`, and how it's used for on-target testing (`ptest-runner`). - `RDEPENDS:${PN}-suffix` scoping a runtime dependency to one sub-package rather than the whole recipe.

Recipe 3 — `libcap-ng` (first real `bbclass`: `autotools`)

File: `meta/recipes-support/libcap-ng/libcap-ng.inc`

The big conceptual jump: `do_configure` / `do_compile` / `do_install` **disappear from the recipe entirely**. They're now supplied by `autotools.bbclass` via `inherit autotools`. This is the moment "a `bbclass` provides default tasks" stops being abstract.

What's new here: - `inherit autotools` — no task functions written by hand at all. - `do_configure:prepend()` — inserting code *before* the class-provided task body runs, instead of replacing it. - `EXTRA_OECONF:append:class-target = "..."` — passing extra flags into the class's `./configure` invocation, scoped to the target build only. - `SRC_URI = "git://..." + SRCREV = "..."` — the git fetcher, pinned to an exact commit. - `BBCLASSEXTEND = "native nativesdk"` again, now on top of `autotools`.

By the end of this step you should understand: - `autotools.bbclass`: what its default `do_configure` (`./configure`), `do_compile` (`make`), and `do_install` (`make install DESTDIR=`) do. - The difference between `:prepend` / `:append` (run before/after the existing function body) vs. fully overriding a task by redefining it (as recipes 1–2 did). - Override syntax in general: `VAR:append:class-target`, `VAR:append:class-native` — how BitBake picks which override applies. - Git fetcher basics: `git://...;branch=...;protocol=https` + `SRCREV`.

Recipe 4 — `acl` (stacking classes + package splitting)

File: `meta/recipes-support/attr/acl_2.3.2.bb`

`inherit autotools gettext ptest` — proof that `inherit` just chains classes together; you already know 2 of these 3. The genuinely new idea is one recipe producing **multiple output packages**.

What's new here: - `inherit autotools gettext ptest` — multiple classes stacked in one line. - `PACKAGES += "lib${BPN}"` — prepending a new package name to the package list BitBake will build. - `FILES:lib${BPN} = "${libdir}/lib*${SOLIBS}"` — telling BitBake which installed files go into that extra package.

By the end of this step you should understand: - `gettext.bbclass` : wiring for i18n/translation build dependencies. - That `inherit` is purely additive/ordered — classes don't conflict as long as they don't define the same task differently. - `PACKAGES` and `FILES:<pkg>` : how one `do_install` output tree gets sliced into several `.ipk` / `.deb` / `.rpm` packages by `do_package`. - Why libraries typically end up in their own package (`libfoo`) separate from binaries (`foo`) or headers (`foo-dev`).

Recipe 5 — `libpng` (cross-recipe **DEPENDS** + sysroot staging)

File: `meta/recipes-multimedia/libpng/libpng_1.6.42.bb`

`DEPENDS = "zlib"` — the recipe from step 2 becomes a **build-time dependency** of this one. This is where "sysroot staging" turns from a buzzword into something you can point at.

What's new here: - `DEPENDS = "zlib"` — a real cross-recipe dependency. - `inherit autotools binconfig-disabled pkgconfig ptest` — reuses autotools/ptest (steps 3/2), adds two new small classes. - `PACKAGES += "${PN}-tools" / FILES:${PN}-tools = "..."` — reuses the package-splitting pattern from step 4.

By the end of this step you should understand: - What `DEPENDS` triggers: `zlib` is built first, then its `do_populate_sysroot` task copies its headers/ `.so` / `.pc` files into `libpng`'s private sysroot *before* `libpng`'s `do_configure` runs. - The difference between `DEPENDS` (build-time) and `RDEPENDS` (runtime, seen in step 2) — and why a recipe can need one without the other. - How `pkgconfig.bbclass` and `.pc` files let `./configure` scripts find dependencies (foreshadows step 6). - That package splitting (step 4) generalizes across totally unrelated recipes — it's a mechanism, not a one-off trick.

Recipe 6 — `pkgconfig` (**PACKAGECONFIG** + native builds)

File: `meta/recipes-devtools/pkgconfig/pkgconfig_git.bb`

The tool that made step 5's dependency resolution work, studied directly. First recipe where the **native** variant (runs on your build host) behaves meaningfully differently from the target variant.

What's new here: - `PACKAGECONFIG ??= "glib"` and `PACKAGECONFIG[glib] = "--without-internal-glib,--with-internal-glib,glib-2.0 pkgconfig-native"` — optional-feature flags mapped to configure args and extra `DEPENDS`. - `PACKAGECONFIG:class-native = ""` — the native variant disables the feature entirely. - `do_install:append:class-native () { ... }` — extra install steps only for the native build. - `RPROVIDES:${PN} += "pkgconfig(pkg-config)"` — a virtual runtime provide.

By the end of this step you should understand: - `PACKAGECONFIG` : the standard mechanism for optional build-time features, and its 4-field syntax (`configure-on,configure-off,extra-DEPENDS,extra-RDEPENDS`). - Why `-native` recipes exist at all: tools that must run *on the build host* during other recipes' builds (this is literally what made step 5's `./configure` able to find `zlib`). - `:class-native` / `:class-target` / `:class-nativesdk` as the three standard `BBCLASSEXTEND` variants and when each applies.

Recipe 7 — `json-c` (a second build-system class: `cmake`)

File: `meta/recipes-devtools/json-c/json-c_0.17.bb`

`inherit cmake ptest` — the exact slot `autotools` filled in step 3, now filled by a different class. Comparing `EXTRA_OECMAKE` here against `EXTRA_OECONF` from steps 3/5/6 is what makes "a build-system class just provides `do_configure/do_compile`" fully click.

What's new here: - `inherit cmake ptest` instead of `autotools`. - `EXTRA_OECMAKE = "-DDISABLE_WERROR=ON"` — `cmake`'s equivalent of `EXTRA_OECONF`. - `RPROVIDES:${PN} = "libjson"` — another virtual provide, reinforcing step 6.

By the end of this step you should understand: - `cmake.bbclass` : default `do_configure` (`cmake ...`) and `do_compile` (`ninja / make`), as an alternative to `autotools.bbclass`. - That "which build-system class you inherit" is an implementation detail from BitBake's point of view — the task names and overall task graph stay identical either way.

Recipe 8 — `busybox` (integration: `PACKAGECONFIG` at scale + config fragments)

File: `meta/recipes-core/busybox/busybox.inc` (the one already in your working tree — good moment to write a small `.bbappend` of your own and confirm it lands with `bitbake-getvar busybox -f do_install`)

A big, real-world recipe, but every individual piece is now familiar except one — and that one is the single most important concept to nail before the kernel recipe.

What's new here: - `PACKAGECONFIG` used at real scale (dozens of flags) — same mechanism as step 6, much bigger. - `inherit update-alternatives` , `useradd` , `systemd` — three new small classes for system integration. - **Config built by merging `.cfg` fragments** rather than one static file — this is the exact pattern `linux-yocto` uses for kernel configuration in step 10. - Many other layers ship `busybox.bbappend` files — a good recipe to practice reading/writing an append against.

By the end of this step you should understand: - `update-alternatives.bbclass` : managing symlinks when multiple packages can provide the same binary name (e.g. `/bin/sh`). -

`useradd.bbclass` : declaring users/groups to be created when the rootfs is populated. -

`systemd.bbclass` : installing and enabling unit files, and `SYSTEMD_SERVICE` / `SYSTEMD_AUTO_ENABLE` . -

Config-fragment merging: how a base config plus a set of small `.cfg` /feature fragments get combined into one final build config — remember this shape, it reappears almost unchanged in recipe 10. - How to actually write and verify a `.bbappend` (`FILESEXTRAPATHS` , `SRC_URI +=` , task `:append`) against a real, widely-appended recipe.

Recipe 9 — `u-boot` (deploy.bbclass + Kconfig, the kernel's structural twin)

Files: `meta/recipes-bsp/u-boot/u-boot.inc` , `meta/classes-recipe/uboot-config.bbclass`

Structurally, this is the closest thing to a kernel recipe that isn't one: Kconfig-style `defconfig` selection (echoes step 8's fragment thinking), a full cross-compiler build, and an output that gets **deployed**, not installed into a package.

What's new here: - `uboot-config.bbclass` — machine-specific `defconfig` selection via `UBOOT_CONFIG` / `UBOOT_MACHINE` . - `inherit deploy` → `do_deploy` — a new task that copies build output straight to `${DEPLOY_DIR_IMAGE}` , bypassing normal packaging. - Explicit `${TARGET_PREFIX}` / `CROSS_COMPILE` -style cross-build env, same idea as `${CC}` in step 1 but at full toolchain scale.

By the end of this step you should understand: - `deploy.bbclass` : the `do_deploy` task and `DEPLOY_DIR_IMAGE` , and how it differs from `do_install` → `do_package` (this output isn't meant to be installed on the target as a package; it's consumed directly by image creation/boot). -

Kconfig/ `defconfig` -based configuration as a second config style, alongside the fragment-merge

style from step 8 — the kernel in recipe 10 actually uses *both* ideas together. - Why a bootloader recipe and a kernel recipe look so similar: both are "cross-compile a standalone image and deploy it," not "build a package for the target rootfs."

Recipe 10 — `linux-yocto` (the capstone)

Files: `meta/recipes-kernel/linux/linux-yocto_6.6.bb` , `meta/recipes-kernel/linux/linux-yocto.inc`

Every previous concept lands here at once. Read the classes in this order — each is layered on by `inherit` inside `kernel.bbclass` /the recipe:

1. `kernel.bbclass` — [meta/classes-recipe/kernel.bbclass](#) Core `do_compile` / `do_install` , and the hand-off point to module packaging.
2. `kernel-arch.bbclass` — [meta/classes-recipe/kernel-arch.bbclass](#) Maps OpenEmbedded's `TARGET_ARCH` to the kernel's own `ARCH` / `CROSS_COMPILE` naming.
3. `kernel-yocto.bbclass` — [meta/classes-recipe/kernel-yocto.bbclass](#) The Yocto-specific kernel tooling: `do_kernel_metadata` , `do_validate_branches` , `do_kernel_configcheck` . This is `KBRANCH` / `SRCREV_machine` (git fetcher, step 3) plus `.scm` config-fragment stacking via `KERNEL_FEATURES:append` — the same fragment-merge idea from step 8, generalized and formalized.
4. `kernel-devicetree.bbclass` — `.dtb` compilation and packaging.
5. `kernel-module-split.bbclass` — auto-splits built `.ko` files into per-module packages: the kernel's automated version of the manual `PACKAGES += / FILES:<pkg>` you did by hand in steps 4 and 5.
6. `deploy.bbclass` (reused from step 9) — publishes `Image` / `vmlinux` / `dtb` files to `DEPLOY_DIR_IMAGE` exactly like `u-boot` did.

By the end of this step you should be able to explain, without looking anything up: - Why `linux-yocto_6.6.bb` has per-machine `KBRANCH:qemuarm` , `SRCREV_machine:qemuarm` , etc. — it's the git-fetcher-plus-override pattern from step 3, scaled to many machines in one recipe. - How `KERNEL_FEATURES:append:qemuall = "... .scm"` assembles the final `.config` — the fragment-merge pattern from step 8, made explicit via `do_kernel_metadata` / `do_kernel_configme` . - Why kernel modules become individual installable packages (`kernel-module-*`) without you writing any `PACKAGES` / `FILES` lines — `kernel-module-split.bbclass` automates exactly what you did manually in `acl` and `libpng` . - Why the recipe's build output ends up in `DEPLOY_DIR_IMAGE` instead of a normal target package — the same `do_deploy` mechanism as `u-boot` . - What `kernel-arch.bbclass` is translating and why cross-compiling the kernel needs its own `ARCH` / `CROSS_COMPILE` mapping distinct from the OE toolchain variables used everywhere else.

bbclass reference table (everything covered above)

Class	What it gives a recipe	First seen at
<code>base.bbclass</code>	Default fetch/unpack/patch tasks every recipe gets, implicitly	Recipe 1
<code>pptest.bbclass</code>	<code>do_install_pptest</code> , a runtime self-test package	Recipe 2
<code>autotools.bbclass</code>	Default <code>do_configure</code> / <code>do_compile</code> / <code>do_install</code> via <code>./configure && make</code>	Recipe 3
<code>gettext.bbclass</code>	i18n/translation build dependency wiring	Recipe 4
<i>(package splitting, <code>package.bbclass</code>)</i>	<code>PACKAGES</code> / <code>FILES:<pkg></code> → multiple output packages from one build	Recipe 4
<i>(staging, <code>staging.bbclass</code>)</i>	<code>DEPENDS</code> → headers/libs populated into your sysroot before configure	Recipe 5
<code>pkgconfig.bbclass</code> / <code>binconfig-disabled.bbclass</code>	<code>.pc</code> file / <code>pkg-config</code> integration for finding dependencies	Recipe 5
<code>native.bbclass</code> (via <code>BBCLASSEXTEND</code>)	A build-host-only variant of a recipe	Recipe 6
<code>cmake.bbclass</code>	<code>cmake</code> -based alternative to <code>autotools.bbclass</code>	Recipe 7
<code>update-alternatives.bbclass</code>	Manages symlink alternatives for conflicting providers	Recipe 8
<code>useradd.bbclass</code>	Declares users/groups created at rootfs population time	Recipe 8
<code>systemd.bbclass</code>	Installs/enables systemd unit files	Recipe 8
<code>deploy.bbclass</code>	<code>do_deploy</code> task, publishes artifacts to <code>DEPLOY_DIR_IMAGE</code>	Recipe 9
<code>uboot-config.bbclass</code>	Machine-specific U-Boot <code>defconfig</code> selection	Recipe 9
<code>kernel.bbclass</code>	Core kernel <code>do_compile</code> / <code>do_install</code> , hand-off to module packaging	Recipe 10
<code>kernel-arch.bbclass</code>	Maps OE arch → kernel <code>ARCH</code> / <code>CROSS_COMPILE</code>	Recipe 10

Class	What it gives a recipe	First seen at
<code>kernel-yocto.bbclass</code>	Branch/fragment-based kernel config tooling	Recipe 10
<code>kernel-devicetree.bbclass</code>	<code>.dtb</code> compilation and packaging	Recipe 10
<code>kernel-module-split.bbclass</code>	Auto-splits built <code>.ko</code> files into per-module packages	Recipe 10

Quick command reference

```
# See the fully resolved shell function for a task, after all classes/appends apply
bitbake-getvar <recipe> -f do_compile
bitbake-getvar <recipe> -f do_configure
bitbake-getvar <recipe> -f do_install

# See every variable's final value and which file set it last
bitbake -e <recipe> | less

# See the full task graph for a recipe
bitbake -g <recipe>          # writes task-depends.dot etc. in cwd

# Check what a running/would-run build actually inherits
bitbake-layers show-recipes <recipe>
```